

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 07 · LECTURE NOTES

Failure Detection and Membership

Deciding who is alive, when you cannot know.

MODULE OUTCOMES

- Explain why perfect failure detection is impossible.
- Compare heartbeat and phi accrual detectors.
- Explain gossip and SWIM and their scaling properties.
- Explain why timeouts must be tuned against tail latency.
- Design membership that tolerates false positives.

1 The impossibility

YOU CANNOT DISTINGUISH DEAD FROM SLOW

A node that has not responded might be crashed, overloaded, paused by its garbage collector or hypervisor, or perfectly healthy but unreachable from you. No observation separates these cases — it is Module 01 §1's ambiguity, now as a design constraint rather than a curiosity.

So a failure detector is necessarily a guess, and it will be wrong in both directions. The design question is not how to be right but how to be wrong safely.

A false positive declares a live node dead. This triggers an unnecessary failover — and, worse, **the 'dead' node is still running and may still be acting**, which is the split-brain hazard of Module 06 §3.

A false negative keeps a dead node in the cluster. Requests routed to it hang until their own timeouts expire, and recovery is delayed by exactly that much. **You tune the trade between these; you do not escape it.**

A TIMEOUT IS A BET ABOUT THE LATENCY TAIL

Too short, and you declare healthy nodes dead every time latency spikes — which it does routinely, from GC, from a scheduler decision, from a noisy neighbour, from a transient network event.

Too long, and real failures go undetected, so requests pile up against a node that is not coming back and the system's recovery time is dominated by the detection delay.

So derive it from the measured latency distribution rather than picking a round number. The p99.9 with a margin is a reasonable starting point, and the right value changes when the workload does — which means it should be re-derived, not set once.

And remember process pauses. A 200 ms heartbeat timeout on a service with 500 ms garbage collection pauses declares failures that are not failures (CSCE 605 Module 12), repeatedly and under load, which is when you can least afford it. **The timeout and the collector's pause target must be chosen together**, and this coupling is routinely missed because the two are configured by different people.

2 Detectors

PHI ACCRUAL: A SUSPICION LEVEL RATHER THAN A BOOLEAN

Record the distribution of heartbeat inter-arrival times and fit a model to it — typically normal or exponential. Then, given the time since the last heartbeat, compute how surprising that silence is under the fitted model.

The output is $\phi = -\log_{10}(\text{probability of a delay at least this long})$. A ϕ of 1 means roughly a 10% chance this is normal; a ϕ of 8 means one in a hundred million. **It is a continuous suspicion level, not a binary verdict.**

So different consumers can apply different thresholds to the same signal. Stop routing new requests at $\phi = 1$ (cheap, easily reversed); trigger an expensive failover at $\phi = 8$. **The action is proportional to the confidence**, which a boolean detector cannot express.

And it adapts automatically. A link that is normally slow and variable produces a wide distribution, so the bar for suspicion rises; a consistently fast link lowers it. **Cassandra and Akka use this**, and it is strictly better than a fixed timeout for essentially no extra cost.

Approach	How it works	Scaling
Central heartbeat	Every node reports to one monitor.	$O(n)$ messages, and the monitor is a single point of failure that must itself be made highly available — which is Module 06.
All-to-all	Everyone pings everyone.	$O(n^2)$ messages per round. Works fine to tens of nodes and collapses somewhere around a hundred.
Gossip	Each node periodically exchanges membership with a random peer , merging by version number.	$O(\log n)$ rounds to reach everyone, with $O(1)$ messages per node per round — see §3.

Approach	How it works	Scaling
SWIM	Probe one random node per round; if it does not answer, ask k other nodes to probe it on your behalf.	$O(1)$ per node per round , with detection time independent of cluster size. The indirect probe is the key idea — it distinguishes 'unreachable from me' from 'dead', which directly attacks the false-positive problem of §1.
Ring / neighbour watching	Each node monitors its successors in a ring.	$O(1)$ per node, and slow to detect the failure of a contiguous region of the ring.
Lease-based	A node holds a time-limited lease and loses authority when it expires , whether or not anyone noticed.	Self-expiring, which is its great virtue — no detection is required for authority to lapse. Pairs with fencing tokens (Module 06 §3).

3 Gossip

```
Every T milliseconds, each node:
  pick a RANDOM peer
  exchange membership tables
  merge: for each node keep the entry with the higher
         version / incarnation number
```

PROPERTIES

- * reaches all n nodes in $O(\log n)$ rounds
- * $O(1)$ messages per node per round, regardless of n
- * no coordinator and no single point of failure
- * robust to loss -- the next round retries implicitly

COST

- * eventual, not immediate: there is a propagation delay
- * $O(n)$ state at every node (the full membership table)
- * conflicting entries need version numbers to order

REFUTATION: a node wrongly declared dead broadcasts a HIGHER incarnation number for itself, which beats the death rumour on merge. This is how SWIM survives false positives with no central arbiter.

The refutation mechanism is what makes false positives survivable. Without it, a single mistaken suspicion propagates to the whole cluster and cannot be withdrawn; with it, **the accused node simply says otherwise and its statement wins**, because it holds a higher incarnation number for itself than anyone else can produce. It is a small and elegant mechanism that turns an unrecoverable error into a transient one.

4 Designing for false positives

- **Make failover idempotent and reversible.** A node declared dead that subsequently returns must be able to rejoin cleanly without manual intervention — which is a design property, not an operational procedure.
- **Fence every side effect** (Module 06 §3), so that a wrongly-evicted node resuming after a pause cannot corrupt anything. **This is the single most important item on the list**, because it converts a correctness failure into a rejected operation.
- **Use suspicion rather than a binary verdict** (§2) and act proportionally to confidence.
- **Require a quorum of observers to agree a node is dead**, rather than letting one observer's view trigger action — which is SWIM's indirect probe generalised.
- **And rate-limit failover. A detector that flaps causes far more damage than the failures it detects:** repeated leader elections, repeated rebalancing, and repeated cache invalidation together constitute an outage. **Most failure-detection incidents are the detector overreacting, not the detector missing something**, and that asymmetry should shape the tuning.

IN A GAME, DISCONNECTION IS A PRODUCT DECISION

A client that stops sending might have crashed, lost its connection, closed the game in frustration, or be deliberately exploiting the ambiguity to avoid a loss — pulling the network cable to escape a fight is an old and effective cheat.

The server cannot tell, and unlike an internal cluster it has no quorum of mutually-trusting peers to consult. There is exactly one observer and it has exactly one noisy signal.

So the policy is a design question rather than a technical one: freeze the character in place, hand it to an AI, mark it invulnerable, or remove it immediately — and how many seconds to wait before doing so.

Every choice is exploitable. Freezing lets players escape damage by disconnecting; immediate removal punishes players with genuinely poor connections; AI takeover produces a character behaving unlike its owner. **There is no correct answer, only a stated one** — which is worth recognising as a general pattern: **when the ambiguity is irreducible, the decision moves from engineering to policy**, and the right output of the engineering is a clear statement of the options and their costs.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Das, Gupta & Motivala — SWIM: Scalable Weakly-consistent Infection-style Process Group Membership (free)	The §2 and §3 protocol , including the indirect probe and the refutation mechanism.
Hayashibara et al. — The ϕ Accrual Failure Detector (free)	The §2 detector, from the paper that introduced it.
HashiCorp — Serf and the Lifeguard extensions (free)	SWIM in production , with the modifications needed to make it behave under real-world latency variance.
Chandra & Toueg — Unreliable Failure Detectors for Reliable Distributed Systems (free)	The theory of §1 — what properties a detector must have for consensus to be solvable despite FLP (Module 05).

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement heartbeat failure detection with a fixed timeout.

- 2 **Measure your service's heartbeat inter-arrival distribution** and choose a timeout from the p99.9 rather than a round number.
- 3 **Induce a 500 ms GC pause** and show your detector declares a false failure.
- 4 Implement a phi accrual detector and plot ϕ over time through a real pause.
- 5 **Use two different ϕ thresholds** for two different actions and show the behaviour differs appropriately.
- 6 Implement gossip membership with version numbers.
- 7 **Measure propagation time against cluster size** from 5 to 100 nodes and confirm the $O(\log n)$ behaviour.
- 8 Implement SWIM's indirect probe and **demonstrate it preventing a false positive** caused by a one-way network problem.
- 9 Implement incarnation-number refutation and show a wrongly-accused node recovering.
- 10 **Make your detector flap** by setting the timeout too low under load, and measure the damage the resulting failovers cause.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why can you not distinguish a dead node from a slow one?
- 2 What are the two kinds of detector error and what does each cost?
- 3 How should a timeout be chosen, and what else must it be chosen against?
- 4 What does phi accrual output, and give two advantages over a threshold.
- 5 Compare six detection approaches on scaling.
- 6 What is SWIM's indirect probe for?
- 7 State gossip's properties and costs.
- 8 What is refutation and why does it matter?
- 9 Give five ways to tolerate false positives, and say which matters most.